

Universidad de los Andes
Facultad de Ingeniería
Arquitectura de computadores – ICC3202
Segundo semestre 2026

Proyecto 3

Grupo 18

Cristóbal Aguirre, Ariam Astudillo, Antonio Bouchon

Este trabajo adapta el Pochoco SoC del Proyecto 2 para implementar un macro digital de celdas estándar con el flujo LibreLane y el PDK GF180MCU. Se reemplazan dependencias específicas de FPGA, se implementan dos organizaciones de memoria y se evalúan mediante síntesis, simulación funcional post-síntesis y diseño físico. La corrida final seleccionada usa ROM combinacional más una RAM de datos reducida, con período objetivo de 100 ns y utilización de core objetivo de 40 %. Esta implementación completó el flujo físico y obtuvo DRC de Magic, LVS y violaciones de antena iguales a cero. El informe presenta también los límites del análisis: el DRC de KLayout fue omitido por falta de runset en la configuración utilizada, permanecen advertencias eléctricas de slew y capacitancia, y el resultado es un macro lógico sin pads ni encapsulado.

1. Introducción y objetivo

El objetivo es llevar el Pochoco SoC que ejecuta el juego de reflejos desde su RTL de FPGA hasta un GDSII de macro digital mediante LibreLane y el PDK GF180MCU. El punto de partida es el RTL, el programa `game.s` y el assembler desarrollados por el grupo para el Proyecto 2. El resultado no corresponde a un chip completo listo para conectarse directamente a una placa: sus puertos son terminales lógicos que requerirían pads, protección y circuitos de manejo en un sistema fabricado.

La corrida final de referencia utiliza `rom_100_40`: período de reloj objetivo 100 ns (10 MHz) y utilización de core objetivo de 40 %. Las comparaciones de memoria, barridos de timing y utilización se analizan por separado. Una corrida se considera cerrada solo si termina el flujo, cumple setup y hold con WNS no negativo y TNS igual a cero en los corners evaluados, y supera las comprobaciones físicas requeridas.

2. Del RTL de FPGA al RTL de ASIC

2.1. Módulo superior y terminales

Se creó `asic_top.v` como módulo superior del macro. Expone reloj, reset activo bajo, cuatro botones, cuatro LED, dos displays de siete segmentos y las señales SPI. El módulo instancia el SoC

y conecta sus puertos sin incluir primitivas de entrada/salida propias de una FPGA. Los terminales representan conexiones del macro a un sistema mayor; no incorporan pads, protección ESD ni drivers para cargas externas.

2.2. Dependencias específicas de FPGA identificadas

La auditoría del RTL original produjo dos errores de módulo faltante (`MODMISSING`) para la primitiva Ice40 `SB_RAM40_4K`, ubicada en el register file. El flujo se detuvo en lint antes de síntesis. La auditoría no generó un diagnóstico independiente para todas las demás construcciones; por eso la tabla distingue la evidencia explícita de los problemas inferidos a partir de la arquitectura original.

Cuadro 1: Dependencias del diseño original y adaptación para ASIC.

Construcción original	**Evidencia o limitación**	**Adaptación realizada**
`SB_RAM40_4K` en el register file	LibreLane/Verilator no encuentra la primitiva Ice40; el log reporta dos errores de módulo faltante.	Se implementa el register file con flip-flops de celdas estándar y lecturas registradas.
`initial` y `$readmemh` para cargar el programa	No se obtiene un mensaje separado en la auditoría, que termina antes al faltar la primitiva de RAM. Un archivo de simulación no es un mecanismo de arranque garantizado en silicio.	El assembler genera una ROM sintetizable con constantes explícitas; la variante unificada carga el programa mediante reset explícito de sus flip-flops.
Inferencia de BRAM/RAM de FPGA	No se asume que una memoria de bloque específica exista en el PDK GF180MCU.	La memoria de datos se reduce según el uso real del juego; la alternativa unificada se implementa con flip-flops.
Estado inicial de encendido en FPGA	Un contador de reset inicializado por `initial` no garantiza un estado equivalente en ASIC.	Se usa reset externo activo bajo; el diseño requiere mantenerlo al menos cinco ciclos y liberarlo de manera sincronizada.
Reloj de 25 MHz	Es un supuesto temporal del programa original, no un error de LibreLane.	Se recalculan las constantes temporales para el período final de 100 ns.

El register file conserva 15 registros de 32 bits y salidas registradas para mantener la latencia de lectura esperada por el core. El registro `x0` permanece en cero y las escrituras dirigidas a él no alteran el estado. Los sincronizadores de botones y SPI se mantienen. El contador libre de ciclos se separa en `cycle_counter.v` para facilitar su identificación y análisis de área.

2.3. Puertos lógicos del macro

La interfaz externa se diseñó para el sistema del juego: `clk_i`, `rst_ni`, `buttons_i`, `leds_o`, `segment1_o`, `segment2_o` y las señales SPI. En una implementación de chip completo, cada señal se conectaría a la infraestructura de pads correspondiente. El GDSII del proyecto representa el macro digital, no dicha infraestructura.

3. Memoria, programa del juego y extensión del assembler

3.1. Uso real del programa

El assembler procesa `sw/game.s` y genera 99 palabras de instrucciones de 32 bits, equivalentes a 396 bytes. El juego no utiliza palabras de RAM de datos: el registro `x2` apunta a la región MMIO y las variables permanecen en registros. Se reserva una palabra de datos para implementar y comprobar el camino de memoria, situada en la dirección `0x18c`. La memoria unificada contiene, por tanto, 100 palabras en total. Estas cantidades se obtienen del análisis del programa ensamblado y quedan registradas en `rtl/generated/program.json`.

3.2. Estrategias comparadas

La primera estrategia implementa una ROM combinacional generada automáticamente desde el programa y una RAM de datos reducida de una palabra. La salida de lectura se registra para conservar el ciclo de respuesta esperado por el core. Las escrituras no modifican las instrucciones de ROM y las direcciones fuera de rango no se pliegan sobre direcciones válidas.

La segunda estrategia implementa 100 palabras escribibles con flip-flops: 99 palabras de programa y una palabra de reserva. El reset carga explícitamente el contenido inicial y las escrituras posteriores usan byte enables. La estrategia representa el costo de mantener instrucciones y datos dentro de una memoria unificada escribible.

Cuadro 2: Comparación de las dos estrategias en GF180MCU.

Estrategia	**Área de síntesis** (μm^2)	**Flip-flops**	**Utilización lógica**	**Fmáx verificada**
ROM + RAM reducida	163 731.187	914	47.79 %	15,873 MHz
Memoria unificada FF	597 386.362	4 083	50.64 %	10,000 MHz

El área tabulada corresponde al área de celdas estándar mapeadas durante síntesis, no al área del die. La memoria unificada requiere aproximadamente 3.65 veces el área de síntesis de ROM+RAM. El cambio principal se explica por almacenar el programa como estado escribible en flip-flops. La Fmáx es la mayor frecuencia demostrada por los puntos de período ensayados que cumplieron el criterio de cierre; no se extrapola a partir de un único slack.

3.3. Extensión del assembler

El assembler se amplió para generar, además del archivo hexadecimal, la imagen ASIC `rtl/generated/asic_memory.v` y un manifiesto de programa con símbolos, cantidad de instrucciones y datos, constantes temporales y hash del código fuente. La generación final se reproduce con:

```
python3 tools/assembler.py sw/game.s sw/game.hex --asic-output rtl/generated
```

Listing 1: Generación de la imagen final desde el programa fuente.

La variante de simulación acorta las esperas para acelerar el testbench. Se genera separadamente y se utiliza la misma imagen reducida al comparar RTL y netlist, evitando comparar programas con constantes distintas.

4. Síntesis y trazabilidad jerárquica

La síntesis se realizó con Yosys usando la biblioteca `gf180mcu_fd_sc_mcu7t5v0`. Se conserva la jerarquía mediante `SYNTH_HIERARCHY_MODE=deferred_flatten`; luego, la atribución por nombres de instancia permite asociar celdas y áreas a bloques del SoC.

4.1. Celdas más utilizadas

Cuadro 3: Cinco tipos de celda con más instancias en la síntesis ROM+RAM.

Celda GF180	Cantidad	Interpretación funcional principal
`dffrnq_1`	911	Registros del core, estado del periférico y memoria de datos/lectura.
`nand2_1`	902	Decodificación de ROM, selección y lógica de control.
`mux2_2`	710	Selección de operandos y datos, principalmente en el register file.
`aoi22_1`	553	Lógica combinacional de selección y habilitación, especialmente en el register file.
`nor2_1`	352	Decodificación y comparaciones en memoria, periféricos y ALU.

La explicación de una celda no depende únicamente de su nombre: el conteo por tipo se contrastó con el prefijo jerárquico de cada instancia en el netlist. La suma de áreas por instancia se realizó con el atributo `area` de la biblioteca Liberty y se comparó con las estadísticas de síntesis.

4.2. Atribución de área

Cuadro 4: Área de síntesis atribuida a bloques seleccionados.

Bloque	Área (μm^2)	Fracción del total
Diseño completo	163 731.187	100.00 %
Core	105 600.096	64.50 %
Register file	71 587.667	43.72 %
ALU	10 152.800	6.20 %
Memoria	25 815.552	15.77 %
Contador de ciclos	3 942.579	2.41 %

Las áreas de register file y ALU están contenidas dentro del área del core, por lo que no deben sumarse nuevamente al total. El register file es el submódulo dominante: requiere almacenamiento y lógica de selección para sus puertos de lectura. El contador de ciclos representa 2.41 % del área de síntesis; no domina el resultado. La comparación entre memorias confirma la expectativa de que una implementación completamente escribible con FF es mayor que una ROM de constantes.

5. Timing: barrido, ruta crítica, optimización y constantes

5.1. Criterio y barrido de período

El cierre requiere que setup y hold cumplan WNS ≥ 0 y TNS $= 0$. La revisión se amplió a nueve corners: RC mínimo, nominal y máximo, cruzados con los modelos típico, lento y rápido del PDK. La tabla presenta los corners típico `nom_tt_025C_5v00` y lento `max_ss_125C_4v50`. WS es el slack reportado; WNS es cero cuando todas las rutas cumplen y negativo cuando existe una violación.

Cuadro 5: Resultados representativos del barrido de timing. Todos los valores están en ns.

Configuración / corner	WNS setup	TNS setup	WNS hold	TNS hold
ROM, 100 ns / típico	0	0	0	0
ROM, 100 ns / lento	0	0	0	0
ROM, 80 ns / lento	0	0	0	0
ROM, 63 ns / lento	0	0	0	0
ROM, 62 ns / lento	-0.657	-4.349	0	0
FF, 100 ns / típico	0	0	0	0
FF, 100 ns / lento	0	0	0	0
FF, 60 ns / lento	-33.757	-52 713.816	0	0

La implementación ROM+RAM cierra en los puntos de 100, 80 y 63 ns; el punto de 62 ns no cierra por setup en el corner lento. Su menor período cerrado entre los ensayados es 63 ns, equivalente a una Fmáx verificada de 15,873 MHz. La referencia FF `ffq_100_40` cierra a 100 ns, pero la prueba a 60 ns se rechaza. Por ello, su Fmáx verificada entre los puntos evaluados es 10 MHz. Cada cambio de período provoca una nueva optimización y puede modificar celdas, buffers y placement.

5.2. Ruta crítica

En la corrida ROM final, la ruta crítica del corner lento parte de la salida registrada de instrucción `u_soc.u_ram._2300_ / Q` y termina en `u_soc.u_periph._1280_ / D`, correspondiente al bit 18 de `elapsed_q`. La ruta atraviesa decode, selección de operandos, ejecución en la ALU para formar la dirección de acceso, selección MMIO y control del periférico hasta capturar el tiempo de respuesta. Su slack de setup mínimo es 37,342.553 ns.

5.3. Cambio RTL evaluado

Se evaluó una suma carry-select por bloques de 8 bits para acelerar la propagación del acarreo. El área sintetizada cambió de 163 731.187 a 163 671.917 µm^2, una reducción aproximada de 0.04 %. Sin embargo, el slack de setup empeoró de 37.343 a 35.564 ns. La ruta incluye decode, selección y control MMIO además de la suma; acelerar una operación local no garantiza mejorar la ruta completa. Por esa razón se conserva la ALU de referencia.

5.4. Constantes temporales del juego

Para mantener las esperas con período de reloj $T = 100\,\text{ns}$, se recalculan los ciclos mediante $N = t/T$:

Cuadro 6: Constantes del programa final para el reloj de 10 MHz.

Constante	Tiempo deseado	Ciclos a 100 ns
`WAIT_CYCLES`	3 s	30 000 000
`TENTH_CYCLES`	0.1 s	1 000 000
`HOLD_CYCLES`	1 s	10 000 000
`RELEASE_CYCLES`	0.02 s	200 000

Las esperas de simulación utilizan valores acortados y documentados; las constantes anteriores pertenecen a la imagen final de operación. El contador de 32 bits se desborda aproximadamente cada 429.5 s a 10 MHz, por lo que las mediciones deben mantenerse dentro de ese intervalo.

6. Floorplan, alimentación, placement y costo

Se evaluaron cuatro valores de `FP_CORE_UTIL`. Al aumentar la utilización objetivo, disminuye el die observado para las corridas ROM, aunque cambian geometría, rutas y congestión. La columna de violaciones indica el conteo reportado en la primera iteración de la última invocación del ruteador detallado; no es el DRC final, que se revisa por separado.

Cuadro 7: Barrido de utilización para ROM+RAM a 100 ns.

Core objetivo	Ocupación lógica	Área de die (µm²)	DRC iteración 0	Cierre final
30 %	35.90 %	579 292	191	Sí
40 %	47.79 %	438 409	95	Sí
50 %	59.98 %	353 522	46	Sí
60 %	71.53 %	296 710	3 618	Sí

El menor die observado entre estos cuatro puntos corresponde a 60 %, con 296 710 µm²; no se afirma que sea el óptimo global. La corrida final conserva 40 % porque ese es el objetivo fijado por el enunciado. La métrica de congestión no es monótona en este barrido: una ocupación mayor puede reducir área y, al mismo tiempo, aumentar el trabajo inicial de ruteo.

Los terminales se ordenaron por función: reloj y reset al sur, SPI al oeste, botones y LED al este, y displays al norte. La red de alimentación utiliza rails Metal1 de 0,6 µm y straps Metal4/Metal5 de 1,6 µm; no se implementa un anillo de pads. La conectividad de VDD y VSS se comprueba en el flujo.

Para el ejercicio de costo lineal se usa el área de die final, no el área de celdas:

$$C = 0.438409\ \text{mm}^2 \times \frac{\$23\,000}{\text{mm}^2} = \$10\,083.41.$$

Esta aproximación excluye pads, encapsulado, máscaras, test, rendimiento de fabricación y costos fijos. La decisión con mayor efecto observado en área es la organización de la memoria: almacenar instrucciones como FF aumenta de forma importante el área respecto de la ROM.

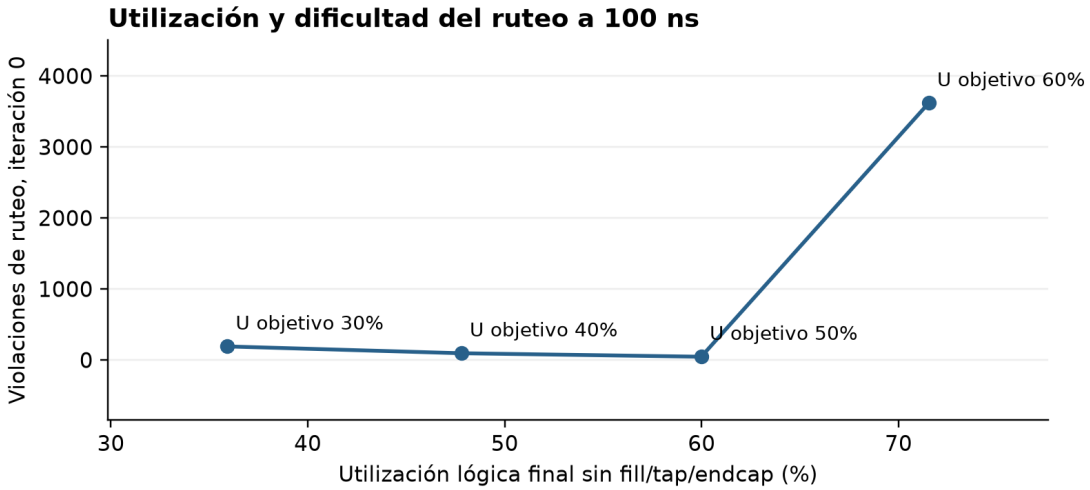


Figura 1: Utilización objetivo y métrica de congestión observada en el barrido.

7. Signoff y layout anotado

La corrida final `rom_100_40` completó LibreLane con cero errores de DRC de Magic, cero errores de LVS de Netgen y cero violaciones finales de antena. El DRC de ruteo también terminó en cero. El PDK no proporciona un runset de KLayout DRC para esta configuración; por tanto, esa comprobación fue omitida y no se contabiliza como aprobada. KLayout sí se utilizó para stream-out, render y comparación XOR.

Durante el flujo se observó una incidencia de antena en la red del sincronizador de MOSI. El reporte previo indicó una razón parcial de 460.48 frente al límite 400. Se insertó una celda de diodo de antena y la verificación final quedó en cero. La imagen siguiente muestra la localización anotada.

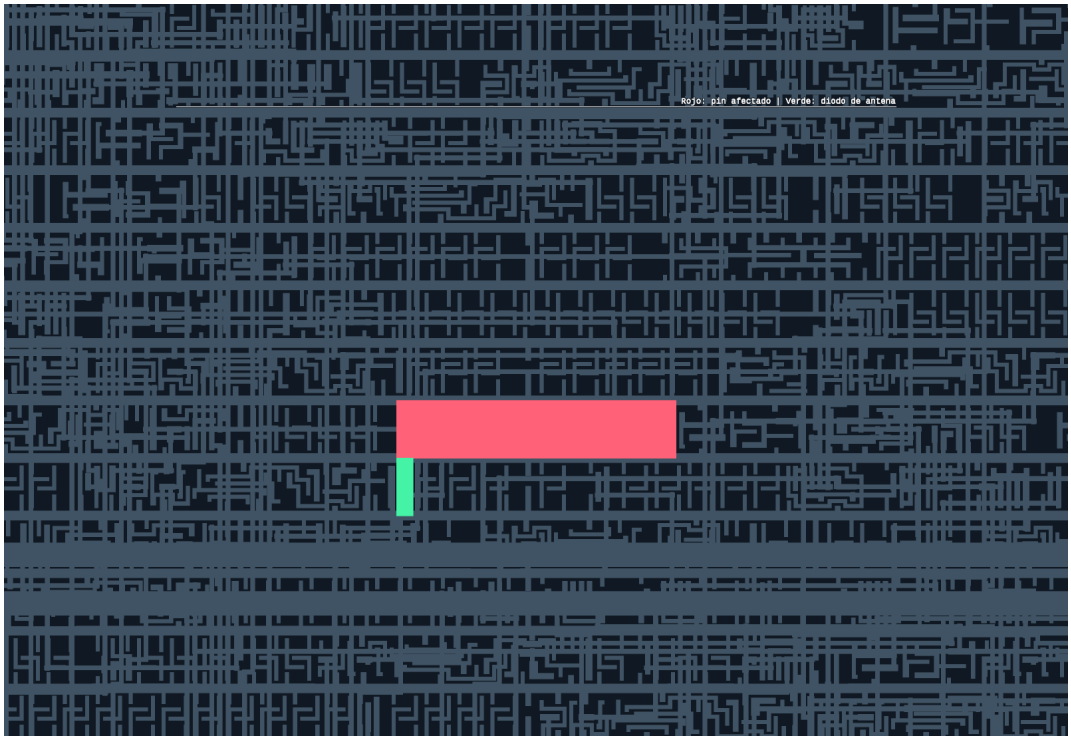
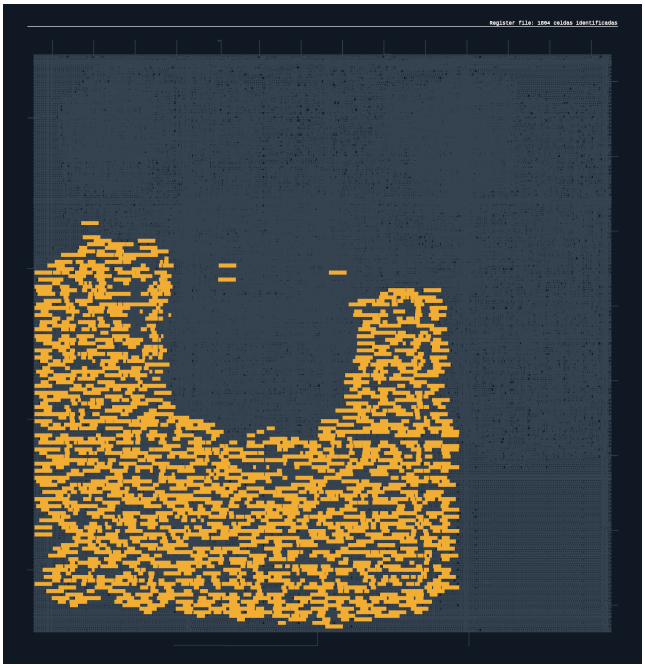


Figura 2: Incidencia de antena y celda de reparación en el layout final.

Las capturas se generaron sobre el GDSII final y utilizan nombres jerárquicos conservados para resaltar el register file, la ALU y el contador. Las superposiciones son anotaciones visuales; no modifican el GDSII.



(a) Register file: `u_soc.u_core.u_rf`.



(b) ALU: `u_soc.u_core.u_id.u_alu`.



(c) Contador: `u_soc.u_periph.u_cycle_counter`.

Figura 3: Bloques del SoC localizados en el layout final.

El GDSII final está en `results/physical/rom_100_40/final/asic_top.gds`. El macro no incluye pads de I/O ni protección ESD. El flujo también reporta advertencias agregadas de slew y capacitancia; se conservan explícitamente y no se confunden con DRC, LVS o WNS/TNS. Por tanto, el cierre presentado es el exigido por el alcance académico, no una certificación de aptitud para fabricación industrial.

8. Simulación funcional post-síntesis

Se simuló el RTL y el netlist mapeado con la biblioteca GF180MCU, sin retardos SDF. El testbench aplica los mismos estímulos en ambos modelos y compara las trazas de LED y displays en cada ciclo observado. También comprueba directamente los ciclos capturados por `elapsed_q` en RTL y en el netlist, usando los nombres preservados.

La prueba cubre reset, espera inicial, respuesta incorrecta, diez rondas correctas, variación de objetivos, presentación del promedio y reinicio. La pulsación incorrecta debe mostrar el código de error, apagar los LED y repetir la ronda sin contarla como acierto. La traza funcional RTL/netlist coincide durante 89 186 ciclos para ROM+RAM y para memoria unificada. En la primera respuesta correcta, el testbench compara una captura de 1 503 ciclos en ambas implementaciones.

Además, las pruebas de memoria verifican escritura, byte enables, reset, protección de ROM y límites de dirección. Las dos variantes de ALU se probaron con 32 000 vectores cada una. La ejecución post-síntesis no reemplaza el análisis estático multicorner y no se realizó simulación post-PnR con SDF.

8.1. Archivos de código y evidencia

El código reproducible que acompaña este informe incluye, entre otros:

- `rtl/asic_top.v`, `rtl/pochoco_soc.v` y módulos del core y periféricos;
- `rtl/generated/asic_memory.v`, generado por `tools/assembler.py`;
- `sw/game.s` y `tools/assembler.py`;
- `tests/tb_asic.v`, `tests/tb_memory.v` y `tests/tb_alu.v`;
- `tools/verify.py`, `tools/post_synthesis.py`, `tools/run_mapped_wsl.sh` y `tools/run_mapped_ff_w`
- las evidencias `results/mapped_verification.json` y `results/mapped_verification_ff.json`.

9. Conclusiones

El SoC se adaptó desde un flujo orientado a FPGA a un macro ASIC sintetizable en GF180MCU. La comparación confirma el costo de área de una memoria unificada con flip-flops y permite seleccionar ROM+RAM para la corrida final de 100 ns y 40 %. La referencia ROM+RAM cerró en setup y hold en los nueve corners revisados, completó DRC de Magic, LVS y reparación de antena, y su comportamiento post-síntesis coincidió con RTL en el testbench. La alternativa FF también tiene una corrida física cerrada a 100 ns, con mayor área.

Los resultados deben interpretarse dentro de su alcance: el flujo no incluye pads ni encapsulado, KLayout DRC se omitió por falta de runset, persisten advertencias eléctricas informadas y la simulación funcional no usa SDF. Las decisiones reportadas se sustentan en las corridas y archivos adjuntos; no se extrapola el punto de frecuencia máxima ensayado a un óptimo continuo.